

Abstraction Before Representation: Deterministic Replay, Semantic Equivalence, and the Ontology of Intelligence

Flyxion

December 16, 2025

Abstract

Contemporary debates about intelligence, abstraction, and understanding are often framed in terms of representation, optimization, or statistical compression. While these approaches have yielded practical successes, they obscure a deeper structural question: what must a system be like in order for abstraction to be possible at all? In this paper, we argue that abstraction is not a matter of representation choice or descriptive economy, but of causal structure. We develop a memory-first, event-based framework in which abstraction arises through deterministic replay, semantic equivalence, and stability under perturbation. Compression, on this view, is neither sufficient nor identical to abstraction. Instead, abstraction corresponds to the formation of equivalence classes over construction histories—structures that permit future reuse, counterfactual reasoning, and semantic invariance. We show that this framework resolves long-standing confusions between memorization and understanding, avoids the limitations of language-centric reasoning models, and provides a principled substrate-neutral account of intelligence applicable across biological and artificial systems.

1 Introduction

What distinguishes genuine understanding from mere compression? This question has re-emerged with renewed urgency in light of recent advances in machine learning systems that achieve remarkable performance while exhibiting fragile generalization, shallow abstraction, and unreliable reasoning. These systems excel at compressing vast corpora of data into compact parameterizations, yet their failures suggest that compression alone is insufficient to account for intelligence.

Philosophical discussions of intelligence have traditionally focused on outward behavior, internal representations, or normative rationality. From Turing’s imitation game to symbolic accounts of cognition and contemporary optimization-based frameworks, intelligence is often characterized by what a system does rather than by how its internal structure supports abstraction. However, such approaches presuppose an ontology—of symbols, goals, or states—that is rarely interrogated.

This paper advances a different starting point. Rather than asking what intelligence represents, we ask what structural conditions must hold for abstraction to be possible at all. We argue that

abstraction is grounded not in representation but in replayable construction. A system can only abstract if it can stably identify when different processes, experiences, or constructions are equivalent for its future behavior. This requires a notion of memory that is not merely storage, but causal trace: a record of how semantic structure was assembled.

Recent work in the mathematical theory of intelligence has emphasized two principles: parsimony and self-consistency. Parsimony captures the intuition that intelligent systems discover low-dimensional structure in the world, while self-consistency demands that internal models remain coherent enough to support prediction and simulation. These principles are compelling, but they leave open a critical distinction: the difference between compression and abstraction. A system may compress without abstracting, and may memorize without understanding.

We propose that this distinction becomes precise when intelligence is analyzed at the level of event structure rather than representation. Compression reduces description length. Abstraction reduces the number of distinct causal representatives required to produce equivalent semantic effects. The former is statistical; the latter is structural. Crucially, abstraction is visible only when a system maintains a replayable history of its own construction.

The framework developed here is deliberately pre-linguistic and pre-representational. Natural language is already a product of abstraction and compression layered atop embodied cognition. Systems that operate solely at the level of linguistic patterns therefore inherit compressed structure without access to the processes that produced it. By contrast, abstraction in animals and early cognition emerges prior to language, through repeated interaction with the world and the stabilization of equivalence under perturbation.

The remainder of the paper proceeds as follows. Section 2 clarifies the metaphysical and epistemic commitments of an event-based ontology. Section 3 introduces deterministic replay as a necessary condition for abstraction. Section 4 develops the notion of semantic equivalence as abstraction over construction histories. Section 5 resolves the compression–abstraction distinction and situates it relative to existing philosophical accounts. Section 6 addresses objections concerning representation, indeterminism, and consciousness. We conclude by outlining the implications of this framework for theories of intelligence and artificial cognition.

2 Ontological Commitments: Events Before States

Most theories of intelligence implicitly adopt a state-based ontology. Systems are described as occupying states in some space—symbolic, probabilistic, or neural—and cognition consists in transitions between these states. While this picture is mathematically convenient, it obscures an important distinction between what a system *is* at a moment and how it came to be that way.

An event-based ontology reverses this emphasis. Instead of treating states as primitive, it treats events—irreversible, causally ordered transitions—as fundamental. States, on this view, are not ontological givens but derived summaries of event histories. This shift has significant consequences for how abstraction is understood.

Definition 2.1 (Event). *An event is a primitive, causally ordered transition that contributes to the construction of semantic structure. Events are irreducible in the sense that they are not defined by their outcomes*

alone, but by their position in a causal history.

Definition 2.2 (Replay). *Replay is the deterministic reconstruction of a system’s semantic state by reapplying its event history from an initial condition.*

Deterministic replay does not imply determinism at the level of the external world. Rather, it requires that once an event has occurred, its contribution to semantic structure is fixed. Replayability ensures that the system can revisit its own construction history, either explicitly or implicitly, and thereby evaluate equivalence across contexts.

This commitment is ontological in a minimal sense: it specifies what kinds of entities must exist for abstraction to be possible. Events are not posited as metaphysical ultimates, but as explanatory primitives justified by their role in grounding abstraction. A purely state-based ontology lacks the resources to distinguish between structures that happen to be similar and structures that are equivalent because they were constructed in equivalent ways.

The importance of this distinction becomes apparent when considering learning systems that achieve impressive compression without robust abstraction. Such systems may converge to similar states via entirely different paths, with no internal mechanism for recognizing those paths as equivalent. Without replay, similarity is accidental; with replay, equivalence can be stabilized.

At this point, a natural objection arises: why insist on deterministic replay rather than probabilistic reconstruction or approximate memory? We address this concern in the next section by arguing that abstraction requires not perfect recall, but causal fidelity.

3 Deterministic Replay and the Conditions of Abstraction

The claim that abstraction requires replay may appear overly strong, particularly in light of biological systems that operate under noise, approximation, and uncertainty. However, the requirement is not that replay be implemented as a literal re-execution of prior events with perfect fidelity. Rather, the requirement is structural: the system must preserve enough causal information about its own construction to determine when different histories are equivalent for future semantic purposes.

Deterministic replay should therefore be understood as a limiting case that clarifies the necessary conditions for abstraction. A system that cannot, even in principle, reconstruct the consequences of alternative construction paths cannot establish semantic invariance across them. Approximate recall may suffice for behavior, but abstraction requires a stronger notion of identity: the ability to recognize that two processes, despite surface variation, license the same future inferences and actions.

This distinction explains why purely statistical or representational systems struggle with abstraction. Such systems may associate similar inputs with similar outputs, but they lack an internal criterion for determining whether those similarities are accidental or principled. Without replay, similarity is observed; with replay, equivalence is constructed.

To see this, consider a system that learns a compressed representation of its inputs. The compression may reduce dimensionality and eliminate redundancy, but it does not by itself specify which distinctions may safely be ignored in future reasoning. Compression optimizes description length;

abstraction optimizes semantic stability. The latter requires knowledge not just of what patterns recur, but of why they recur—knowledge that is accessible only through construction history.

Replay provides the minimal structure needed to answer such questions. By reconstructing the causal sequence that led to a semantic state, the system can evaluate whether alternative sequences would have produced equivalent outcomes. This evaluation is inherently counterfactual: it asks what would have happened had the construction proceeded differently. Without replay, counterfactual reasoning collapses into speculation.

It is therefore not determinism per se that matters, but causal traceability. A system may tolerate noise and uncertainty in execution so long as it preserves a stable mapping from event histories to semantic effects. Deterministic replay formalizes this requirement by making the mapping explicit.

4 Semantic Equivalence as the Core of Abstraction

If replay establishes the conditions under which abstraction is possible, semantic equivalence specifies what abstraction consists in. Abstraction is not the elimination of detail, but the identification of distinctions that no longer matter.

Definition 4.1 (Semantic Equivalence). *Two constructed entities are semantically equivalent if, under all admissible continuations of the event history, they induce indistinguishable semantic effects.*

This definition is intentionally future-oriented. Equivalence is not defined by past similarity alone, nor by present appearance, but by stability of consequence. Two entities may differ in origin, structure, or representation, yet be equivalent if those differences never propagate into meaningful divergence.

Abstraction occurs when the system commits to such equivalence. This commitment is not a discovery of a pre-existing fact, but a structural decision: the system chooses to treat certain distinctions as irrelevant going forward. Importantly, this choice does not erase history. The original construction paths remain recoverable through replay, but their differences are no longer semantically active.

This perspective resolves a persistent confusion in the philosophy of abstraction. Abstraction is often described as a mapping from particulars to universals, or from instances to categories. In the replay-based framework, abstraction is instead a reduction in the number of representatives required to realize a semantic effect. Categories are not primitive; they are induced by equivalence over histories.

The distinction between compression and abstraction becomes precise at this point. Compression reduces the cost of describing or storing construction history. Abstraction reduces the cost of reasoning about the future. A compressed system may still require many distinct representatives to handle future contingencies; an abstract system does not.

This distinction also explains why abstraction is inherently normative. To declare two entities equivalent is to assert that their differences are irrelevant for some purpose. That purpose may be implicit, embodied, or task-relative, but it is always present. Abstraction is therefore inseparable from agency: it reflects what the system cares about preserving.

5 Why Compression Alone Is Insufficient in Principle

It is tempting to identify abstraction with compression, particularly in systems that achieve impressive generalization through statistical learning. However, this identification fails not merely empirically but in principle.

Compression operates on descriptions. It exploits redundancy in data to produce shorter encodings. In doing so, it may discard information deemed irrelevant according to some criterion, often statistical frequency or predictive utility. Yet compression does not, by itself, specify which discarded distinctions will remain irrelevant under future perturbations.

A compressed representation may work well within the distribution on which it was learned, while failing catastrophically outside it. This is not a contingent flaw of current algorithms, but a structural limitation of compression-based abstraction. Compression optimizes for the past; abstraction must constrain the future.

The difference becomes especially clear when considering language. Natural language is already a highly compressed and abstracted medium. Words and grammatical constructions collapse vast swathes of experiential variation into compact symbols. Systems that operate solely on language therefore operate on a signal that has already undergone abstraction elsewhere. They may recombine abstractions, but they do not participate in their formation.

This explains why such systems often exhibit brittle reasoning when confronted with novel situations that violate implicit assumptions encoded in language. The abstractions they manipulate are inherited, not grounded. Without access to the construction histories that gave rise to those abstractions, the system cannot evaluate their stability under perturbation.

By contrast, a replay-based system abstracts by stabilizing equivalence at the point of construction. It does not merely inherit compressed symbols; it constructs its own semantic invariants through interaction. This is why abstraction in animals and early cognition precedes language, and why language amplifies rather than creates abstraction.

The insufficiency of compression is therefore not a matter of scale or data availability. No amount of compressed description can substitute for the ability to identify and stabilize semantic equivalence across construction histories. Abstraction requires structure that compression alone cannot supply.

6 Pre-Linguistic Abstraction and the Animal Case

One of the strengths of a replay-based account of abstraction is that it accommodates forms of intelligence that operate without language. Animals routinely exhibit abstraction: they generalize across contexts, recognize functional equivalence, and adapt behavior in ways that transcend specific stimuli. These capacities emerge despite the absence of symbolic representation in the human sense.

From the present perspective, this is unsurprising. Abstraction does not require symbols; it requires stabilized equivalence under replay. Animals interact with the world through repeated sensorimotor loops, accumulating construction histories that encode causal regularities. Through these interactions, certain distinctions prove irrelevant for survival, while others remain crucial.

Abstraction consists in committing to this irrelevance.

Language enters later as an externalization of abstraction. It provides a shared medium for stabilizing and transmitting equivalence across individuals and generations. But it presupposes abstraction rather than generating it. A system that begins with language but lacks replayable construction history inherits abstractions without understanding their grounds.

This analysis also clarifies the relationship between abstraction and learning. Learning, in the abstract sense, is not merely parameter adjustment but the restructuring of equivalence classes. A system learns when it revises which distinctions it treats as semantically significant. Replay provides the mechanism by which such revisions can be evaluated and stabilized.

At this point, the framework may appear to verge on circularity: abstraction requires replay, replay presupposes memory, and memory seems to require abstraction to be useful. The resolution lies in recognizing that abstraction is not an all-or-nothing phenomenon. Primitive forms of equivalence emerge early and are refined over time. Replay scaffolds abstraction, which in turn guides further replay.

7 Representation Revisited

The preceding analysis does not entail a rejection of representation. Rather, it situates representation within a broader semantic architecture. Representations are stabilized projections of abstracted structure. They are useful precisely because abstraction has already occurred.

In representational systems that lack replay, representations become opaque. They encode correlations without grounding, and their semantic scope is fixed by training rather than by structural invariance. In systems with replay, representations can be revised, inspected, and reinterpreted because their relation to construction history remains accessible.

This distinction allows us to reconcile the utility of representational models with their limitations. Representations are not the foundation of abstraction; they are its artifacts. Understanding this ordering is crucial for avoiding category errors in theories of intelligence.

8 Conclusion

We have argued that abstraction is not reducible to compression, representation, or statistical generalization. Instead, abstraction arises from the stabilization of semantic equivalence under replayable construction histories. Deterministic replay provides the minimal structure required to evaluate and commit to such equivalence, while compression optimizes description without guaranteeing semantic invariance.

This framework resolves long-standing confusions between memorization and understanding, explains the pre-linguistic nature of abstraction in animals, and clarifies why language-centric systems inherit rather than generate abstraction. More broadly, it suggests that intelligence is grounded not in expressive power alone, but in the disciplined organization of memory, causality, and equivalence.

By shifting attention from representations to construction, and from description to replay, we obtain a structural account of abstraction that is both philosophically principled and applicable across domains. Whether instantiated in biological organisms or artificial systems, abstraction requires the same fundamental ingredients: memory that preserves causal trace, mechanisms for replay, and the capacity to stabilize equivalence under variation. Without these, intelligence remains superficial. With them, understanding becomes possible.